Filtering in Pandas 1

Filtering Dataframes

- New dataframe methods
  - Nunique()
  - Duplicated(), drop_duplicates()
  - New inputs to string methods
- Faster way to use apply
  - Defining functions using lambda
- How do I only select row rows that satisfy a certain condition? Multiple conditions?
  - What is going on under the hood?

Using nunique()

```
df
```

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
2	Hugo Lloris	France	OSCM400	2.90	12
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18

```
#Returns the number of unique entries
df.Home_Country.nunique()
```

```
3
```

Can only be applied to single columns

Using duplicated()

Let’s look at how to deal with duplicate columns:

```
df
```

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
2	Hugo Lloris	France	OSCM400	2.90	12
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18
5	Danny Rose	England	QBA200	3.90	17
6	Deandre Yedlin	USA	QBA200	4.00	12

Using duplicated()

Let's look at how to deal with duplicate columns:

```
df
```

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
2	Hugo Lloris	France	OSCM400	2.90	12
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18
5	Danny Rose	England	QBA200	3.90	17
6	Deandre Yedlin	USA	QBA200	4.00	12

```
#On a dataframe
df.duplicated(keep="first")
```

```
0    False
1    False
2    False
3    False
4    False
5     True
6     True
dtype: bool
```

Returns series of boolean with True for all repeated rows.

- keep = "first" – put False for the first occurrence.

Using duplicated()

Let's look at how to deal with duplicate columns:

```
df
```

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
2	Hugo Lloris	France	OSCM400	2.90	12
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18
5	Danny Rose	England	QBA200	3.90	17
6	Deandre Yedlin	USA	QBA200	4.00	12

```
#On a dataframe
df.duplicated(keep="last")
```

```
0    False
1     True
2    False
3     True
4    False
5    False
6    False
dtype: bool
```

Returns series of boolean with True for all repeated rows.

- keep = "last" – put False for the last occurrence

Using duplicated()

Let's look at how to deal with duplicate columns:

```
df
```

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
2	Hugo Lloris	France	OSCM400	2.90	12
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18
5	Danny Rose	England	QBA200	3.90	17
6	Deandre Yedlin	USA	QBA200	4.00	12

```
#On a dataframe
df.duplicated(keep=False)
```

```
0    False
1     True
2    False
3     True
4    False
5     True
6     True
dtype: bool
```

Returns series of boolean with True for all repeated rows.

- keep = False – put True for every row of a repeated occurrence

Using duplicated()

Let's look at how to deal with duplicate columns:

```
df
```

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
2	Hugo Lloris	France	OSCM400	2.90	12
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18
5	Danny Rose	England	QBA200	3.90	17
6	Deandre Yedlin	USA	QBA200	4.00	12

```
#On a single column
df.Home_Country.duplicated(keep="first")
```

```
0    False
1     True
2    False
3    False
4     True
5     True
6     True
Name: Home_Country, dtype: bool
```

Using duplicated()

Let's look at how to deal with duplicate columns:

```
df
```

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
2	Hugo Lloris	France	OSCM400	2.90	12
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18
5	Danny Rose	England	QBA200	3.90	17
6	Deandre Yedlin	USA	QBA200	4.00	12

```
#On a subset of columns
df.duplicated(keep="first", subset = ["Home_Country", "Class"])
```

```
0    False
1     True
2    False
3    False
4    False
5     True
6     True
dtype: bool
```

Using drop_duplicates()

Let's look at how to deal with duplicate columns:

```
df
```

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
2	Hugo Lloris	France	OSCM400	2.90	12
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18
5	Danny Rose	England	QBA200	3.90	17
6	Deandre Yedlin	USA	QBA200	4.00	12

```
#drop duplicates on dataframe
df.drop_duplicates(keep = "first")
```

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
2	Hugo Lloris	France	OSCM400	2.90	12
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18

"Drop all duplicate rows except for the first."

Using drop_duplicates()

Let's look at how to deal with duplicate columns:

```
df
```

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
2	Hugo Lloris	France	OSCM400	2.90	12
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18
5	Danny Rose	England	QBA200	3.90	17
6	Deandre Yedlin	USA	QBA200	4.00	12

```
#drop duplicates on dataframe
df.drop_duplicates(keep = False)
```

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
2	Hugo Lloris	France	OSCM400	2.90	12
4	Joe Cole	England	OSCM400	3.45	18

"Drop all duplicate rows."

Using drop_duplicates()

Let's look at how to deal with duplicate columns:

```
df
```

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
2	Hugo Lloris	France	OSCM400	2.90	12
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18
5	Danny Rose	England	QBA200	3.90	17
6	Deandre Yedlin	USA	QBA200	4.00	12

```
#drop duplicates on dataframe
df.drop_duplicates(keep = "last", subset= ["Home_Country", "Class"])
```

	Names	Home_Country	Class	GPA	Num_Credits
2	Hugo Lloris	France	OSCM400	2.90	12
4	Joe Cole	England	OSCM400	3.45	18
5	Danny Rose	England	QBA200	3.90	17
6	Deandre Yedlin	USA	QBA200	4.00	13

"Drop all duplicate rows based on Home_Country and Class except for the last."

Using drop_duplicates()

Let's look at how to deal with duplicate columns:

```
df
```

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
2	Hugo Lloris	France	OSCM400	2.90	12
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18
5	Danny Rose	England	QBA200	3.90	17
6	Deandre Yedlin	USA	QBA200	4.00	12

```
#drop duplicates on dataframe
df.drop_duplicates(keep = "first", inplace = True)
df
```

Change dataframe in place

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
2	Hugo Lloris	France	OSCM400	2.90	12
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18

New String Method Techniques

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
2	Hugo Lloris	France	OSCM400	2.90	12
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18

How do I add a column for each person's last name?

```
#Method 1: Using map/apply
def Get_Last_Name(name):

    return name.split(" ")[1]

df["Last_Name"] = df["Names"].map(Get_Last_Name)
df
```

	Names	Home_Country	Class	GPA	Num_Credits	Last_Name
0	Harry Kane	England	QBA200	3.78	15	Kane
1	Danny Rose	England	QBA200	3.90	17	Rose
2	Hugo Lloris	France	OSCM400	2.90	12	Lloris
3	Deandre Yedlin	USA	QBA200	4.00	13	Yedlin
4	Joe Cole	England	OSCM400	3.45	18	Cole

New String Method Techniques

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
2	Hugo Lloris	France	OSCM400	2.90	12
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18

How do I add a column for each person's last name?

```
#Method 2: String Methods
df["Name_List"] = df["Names"].str.split(" ")
df
```

	Names	Home_Country	Class	GPA	Num_Credits	Name_List
0	Harry Kane	England	QBA200	3.78	15	[Harry, Kane]
1	Danny Rose	England	QBA200	3.90	17	[Danny, Rose]
2	Hugo Lloris	France	OSCM400	2.90	12	[Hugo, Lloris]
3	Deandre Yedlin	USA	QBA200	4.00	13	[Deandre, Yedlin]
4	Joe Cole	England	OSCM400	3.45	18	[Joe, Cole]

How do I get the last name?

New String Method Techniques

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
2	Hugo Lloris	France	OSCM400	2.90	12
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18

How do I add a column for each person's last name?

```
#Method 2: String Methods - Use expand
df[["First_Name", "Last_Name"]] = df["Names"].str.split(" ", expand = True)
df
```

	Names	Home_Country	Class	GPA	Num_Credits	First_Name	Last_Name
0	Harry Kane	England	QBA200	3.78	15	Harry	Kane
1	Danny Rose	England	QBA200	3.90	17	Danny	Rose
2	Hugo Lloris	France	OSCM400	2.90	12	Hugo	Lloris
3	Deandre Yedlin	USA	QBA200	4.00	13	Deandre	Yedlin
4	Joe Cole	England	OSCM400	3.45	18	Joe	Cole

Creates separate column for each element in the list.

Lambda Functions

Lambda function allows us to create simple function in one line without a def.

```
def f(x,y):
 """Add x and y"""
     return x+y

f(5,6)
```

```
11
```

Lambda Functions

Lambda function allows us to create simple function in one line without a def.

```
def f(x,y):
 """Add x and y"""
     return x+y

f(5,6)
```

```
11
```

Let’s see how we can use a lambda function to do the same thing

Lambda Functions

Lambda function allows us to create simple function in one line without a def.

```
def f(x,y):
 """Add x and y"""
    return x+y

f(5,6)
```

```
11
```

Let's see how we can use a lambda function to do the same thing

```
#Add x and y with lambda
f = lambda x,y: x+y

f(5,6)
```

```
11
```

Lambda Functions

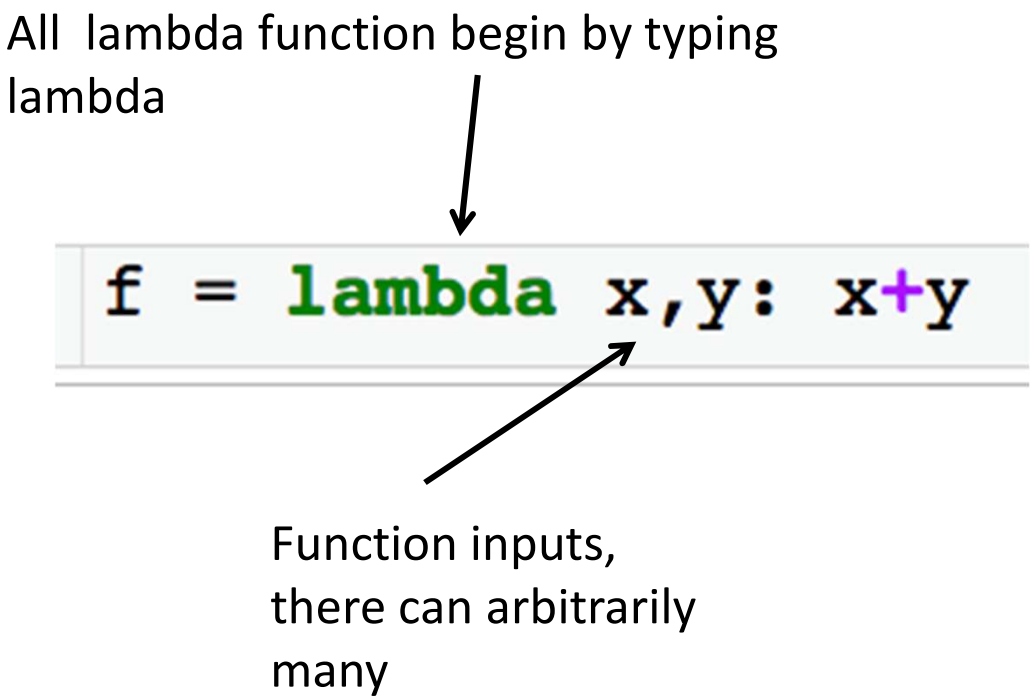
Dissecting the lambda function:

All lambda function begin by typing lambda

```
f = lambda x,y: x+y
```

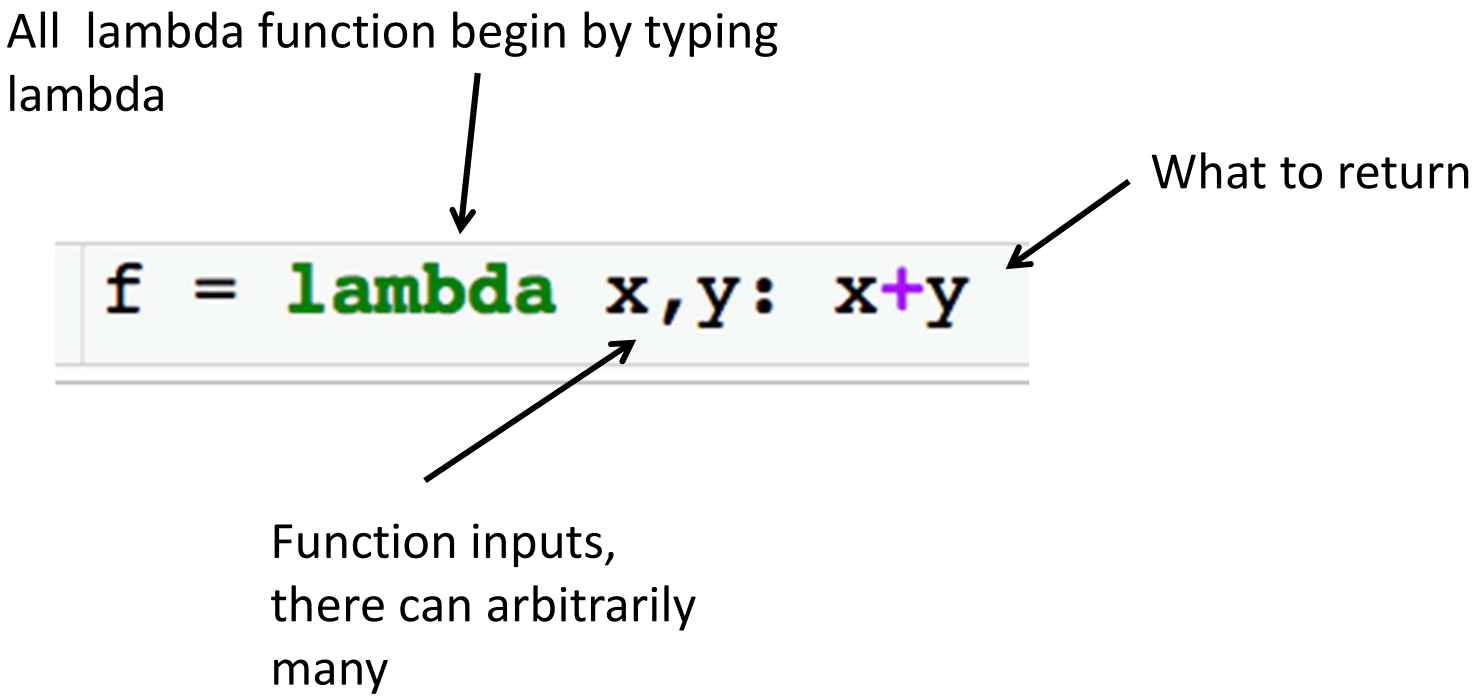
Lambda Functions

Dissecting the lambda function:



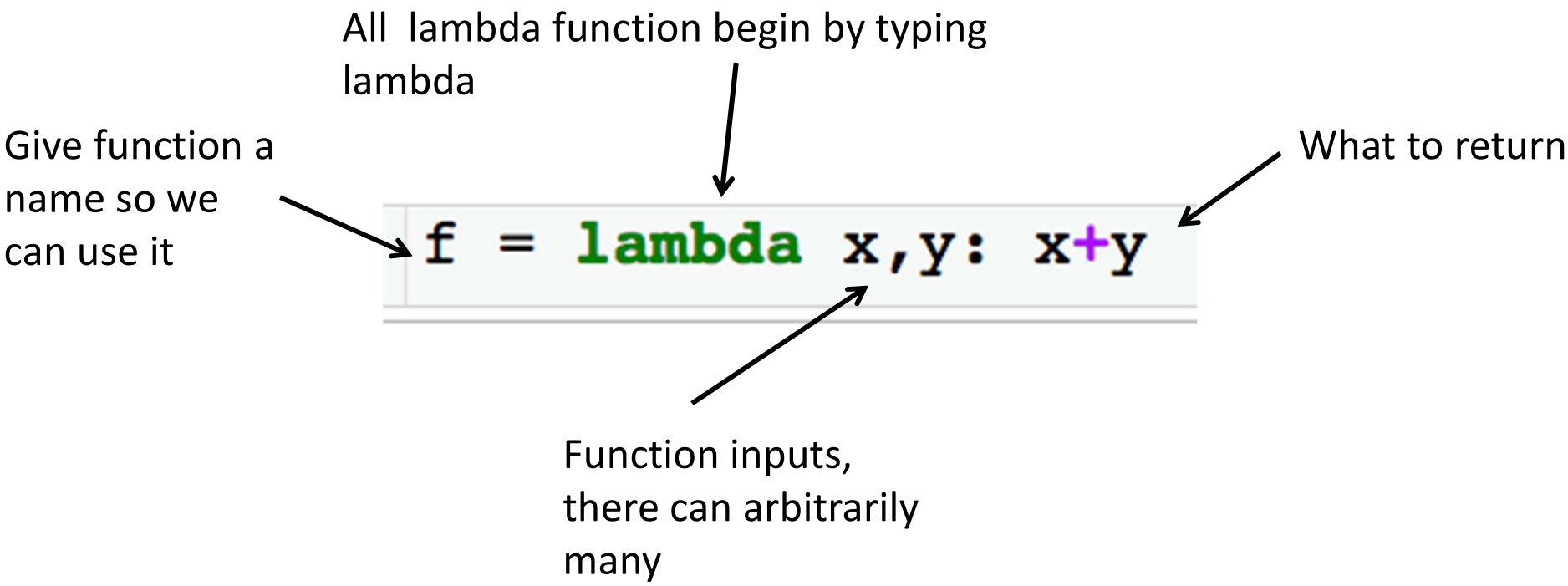
Lambda Functions

Dissecting the lambda function:



Lambda Functions

Dissecting the lambda function:



Lambda Functions

Write a lambda function that returns the sum of the first element and the last element of a list

Lambda Functions

Another example:

```
def f(l):
    """Add first and last elements
    of a list"""
    return l[0] + l[-1]

f([1,2,3,4])
```

Lambda function:

```
#Adds first and last element
f = lambda l: l[0] + l[-1]

f([1,2,3,4])
```

```
5
```

Lambda Functions

Another example:

```
def f(name):
    """Get last name"""
    return name.split(" ")[1]

f("Jake Feldman")
```

```
'Feldman'
```

Lambda function:

```
#Get last name
f = lambda name: name.split(" ")[1]

f("Jake Feldman")
```

```
'Feldman'
```

sorted() in Python

Syntax

```
sorted(iterable, key=key, reverse=reverse)
```

Parameter Values

Parameter	Description
iterable	Required. The sequence to sort, list, dictionary, tuple etc.
key	Optional. A Function to execute to decide the order. Default is None
reverse	Optional. A Boolean. False will sort ascending, True will sort descending. Default is False

Uses of Lambda Functions

Can uses lamba functions as input to key in sorted function:

```
l = [[1,2], [5,3], [1,1], [7,4], [12,1.5]]
#Sorts by first element
sorted(l)
```

```
[[1, 1], [1, 2], [5, 3], [7, 4], [12, 1.5]]
```

```
l = [[1,2], [5,3], [1,1], [7,4], [12,1.5]]
#Sort by second element
```

Uses of Lambda Functions

Can uses lamba functions as input to key in sorted function:

```
l = [[1,2], [5,3], [1,1], [7,4], [12,1.5]]
#Sorts by first element
sorted(l)
```

```
[[1, 1], [1, 2], [5, 3], [7, 4], [12, 1.5]]
```

```
l = [[1,2], [5,3], [1,1], [7,4], [12,1.5]]
#Sort by second element
sorted(l, key=lambda l: l[1])
```

```
[[1, 1], [12, 1.5], [1, 2], [5, 3], [7, 4]]
```

Lambda function takes list and returns the second element

Uses of Lambda Functions

Can uses lamba functions as input to key in sorted function:

```
l = [[1,2], [5,3], [1,1], [7,4], [12,1.5]]
#Sorts by first element
sorted(l)
```

```
[[1, 1], [1, 2], [5, 3], [7, 4], [12, 1.5]]
```

```
l = [[1,2], [5,3], [1,1], [7,4], [12,1.5]]
#Sort by second element
sorted(l, key=lambda l: l[1])
```

```
[[1, 1], [12, 1.5], [1, 2], [5, 3], [7, 4]]
```

Lambda function takes list and returns the second element

With the key parameter you can specify a function to be called on each list element prior to making comparisons. The value of the key parameter should be a function that takes a single argument and returns a key to use for sorting purposes.

Uses of Lambda Functions

Can uses lambda functions as input to key in sorted function:

```
l = [[1,2], [5,3], [1,1], [7,4], [12,1.5]]
#Sorts by first element
sorted(l)
```

```
[[1, 1], [1, 2], [5, 3], [7, 4], [12, 1.5]]
```

```
l = [[1,2], [5,3], [1,1], [7,4], [12,1.5]]
#Sort by second element
sorted(l, key=lambda l: l[1])
```

```
[[1, 1], [12, 1.5], [1, 2], [5, 3], [7, 4]]
```

Lambda function takes list and returns the second element

With the key parameter you can specify a function to be called on each list element prior to making comparisons. The value of the key parameter should be a function that takes a single argument and returns a key to use for sorting purposes.

Use of Lambda Functions

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
2	Hugo Lloris	France	OSCM400	2.90	12
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18

How do I add a column for each person's last name?

```
#Method 1: Using map/apply
def Get_Last_Name(name):

    return name.split(" ")[1]

df["Last_Name"] = df["Names"].map(Get_Last_Name)
df
```

Already wrote this function as a lambda function

	Names	Home_Country	Class	GPA	Num_Credits	Last_Name
0	Harry Kane	England	QBA200	3.78	15	Kane
1	Danny Rose	England	QBA200	3.90	17	Rose
2	Hugo Lloris	France	OSCM400	2.90	12	Lloris
3	Deandre Yedlin	USA	QBA200	4.00	13	Yedlin
4	Joe Cole	England	OSCM400	3.45	18	Cole

Use of Lambda Functions

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
2	Hugo Lloris	France	OSCM400	2.90	12
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18

How do I add a column for each person's last name?

```
#Add last name with map and lambda
df["Last_Name"] = df.Names.map(lambda x: x.split(" ")[1])
df
```

	Names	Home_Country	Class	GPA	Num_Credits	Last_Name
0	Harry Kane	England	QBA200	3.78	15	Kane
1	Danny Rose	England	QBA200	3.90	17	Rose
2	Hugo Lloris	France	OSCM400	2.90	12	Lloris
3	Deandre Yedlin	USA	QBA200	4.00	13	Yedlin
4	Joe Cole	England	OSCM400	3.45	18	Cole

Filtering

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
2	Hugo Lloris	France	OSCM400	2.90	12
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18

Always name_df.loc

```
#Select rows for people with GPA>=3
df.loc[df.GPA>=3,:]
```

only rows where GPA>=3

All columns

Filtering

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
2	Hugo Lloris	France	OSCM400	2.90	12
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18

```
#Select rows for people with GPA>=3
df.loc[df.GPA>=3,:]
```

df[df.GPA>=3]

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18

Index is messed up

Returns a dataframe

Filtering

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
2	Hugo Lloris	France	OSCM400	2.90	12
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18

```
#Select rows for people with GPA>=3
#Only columns Names,Class
df.loc[df.GPA>=3,["Names", "Class"]]
```

	Names	Class
0	Harry Kane	QBA200
1	Danny Rose	QBA200
3	Deandre Yedlin	QBA200
4	Joe Cole	OSCM400

We don't have to select the column we filter on.

df[df.GPA>=3][["Names", "Class"]]

Filtering

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
2	Hugo Lloris	France	OSCM400	2.90	12
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18

```
#Storing the result
df_smart = df.loc[df.GPA>=3,["Names", "Class", "GPA"]]
df_smart
```

	Names	Class	GPA
0	Harry Kane	QBA200	3.78
1	Danny Rose	QBA200	3.90
3	Deandre Yedlin	QBA200	4.00
4	Joe Cole	OSCM400	3.45

Under Hood

Only keep rows where there is a True

```
#Return series of booleans
df.GPA>=3
```

```
0     True
1     True
2    False
3     True
4     True
Name: GPA, dtype: bool
```

```
#GPA>=3
df.loc[df.GPA>=3,:]
```

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18

Filtering – Multiple Conditions

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
2	Hugo Lloris	France	OSCM400	2.90	12
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18

```
#English People with >15 Credits
df.loc[(df.Home_Country == "England") \
       & (df.Num_Credits>15), : ]
```

"and"

Make sure you put each condition in parentheses

Filtering – Multiple Conditions

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
2	Hugo Lloris	France	OSCM400	2.90	12
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18

```
#English People with >15 Credits
df.loc[(df.Home_Country == "England") \
        & (df.Num_Credits>15), : ]
```

	Names	Home_Country	Class	GPA	Num_Credits
1	Danny Rose	England	QBA200	3.90	17
4	Joe Cole	England	OSCM400	3.45	18

df[(df.Home_Country == “England”) & (df.Num_Credits > 15)]

Filtering – Multiple Conditions

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
2	Hugo Lloris	France	OSCM400	2.90	12
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18

```
#Name contains "D" or Class == OSCM400
df.loc[(df.Names.str.contains("D")) \
       | (df.Class == "OSCM400"), : ]
```

"or"

Returns True if string contains a "D"

Filtering – Multiple Conditions

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
2	Hugo Lloris	France	OSCM400	2.90	12
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18

```
#Name contains "D" or Class == OSCM400
df.loc[(df.Names.str.contains("D")) \
       | (df.Class == "OSCM400"), : ]
```

	Names	Home_Country	Class	GPA	Num_Credits
1	Danny Rose	England	QBA200	3.90	17
2	Hugo Lloris	France	OSCM400	2.90	12
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18

Filtering – Multiple Conditions

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
2	Hugo Lloris	France	OSCM400	2.90	12
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18

```
#Either have GPA>=3.5 OR at least 15 credits
#AND be in OSCM400
df.loc[((df.GPA >= 3.5) | (df.Num_Credits>= 15)) \
       & (df.Class == "OSCM400"), : ]
```

What will this give me?

Filtering – Multiple Conditions

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
2	Hugo Lloris	France	OSCM400	2.90	12
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18

```
#Either have GPA>=3.5 OR at least 15 credits
#AND be in OSCM400
df.loc[((df.GPA >= 3.5) | (df.Num_Credits>= 15)) \
        & (df.Class == "OSCM400"), : ]
```

	Names	Home_Country	Class	GPA	Num_Credits
4	Joe Cole	England	OSCM400	3.45	18

Filtering – Multiple Conditions

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
2	Hugo Lloris	France	OSCM400	2.90	12
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18

```
#People from France or USA
df.loc[(df.Home_Country == "France") \
       | (df.Home_Country == "USA"), : ]
```

	Names	Home_Country	Class	GPA	Num_Credits
2	Hugo Lloris	France	OSCM400	2.9	12
3	Deandre Yedlin	USA	QBA200	4.0	13

Filtering – Multiple Conditions

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
2	Hugo Lloris	France	OSCM400	2.90	12
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18

```
df.Home_Country.isin(["France", "USA"])
```

```
0    False
1    False
2     True
3     True
4    False
Name: Home_Country, dtype: bool
```

Input is a list

Filtering – Multiple Conditions

	Names	Home_Country	Class	GPA	Num_Credits
0	Harry Kane	England	QBA200	3.78	15
1	Danny Rose	England	QBA200	3.90	17
2	Hugo Lloris	France	OSCM400	2.90	12
3	Deandre Yedlin	USA	QBA200	4.00	13
4	Joe Cole	England	OSCM400	3.45	18

```
#People from France or USA
#Using isin()
df.loc[df.Home_Country.isin(["France", "USA"]), : ]
```

	Names	Home_Country	Class	GPA	Num_Credits
2	Hugo Lloris	France	OSCM400	2.9	12
3	Deandre Yedlin	USA	QBA200	4.0	13

df[(df.Home_Country == “France”) | (df.Home_Country == “USA”)]